CarLog — Frontend

JaviRSDEV

[Angular-v21] [TypeScript-5.9] [Bootstrap-5.3]
[TailwindCSS-4.1] [Vitest-4.0] [SonarQube-A]

CarLog is a web application designed to digitize the complete vehicle maintenance lifecycle: from workshop registration to detailed billing via work orders. It provides specialized interfaces for three primary user roles: **Managers**, **Mechanics**, and **Clients**.

Contents

1 Project Status **2**

2 Tech Stack **2**

3 Prerequisites **2**

4 Installation **2**

5 Environment Configuration **3**
5.1 Local Development 3
5.2 Production 3

6 Architecture **3**

7 Routes and Navigation **4**

8 Security and Authentication **4**
8.1 Authentication Flow 5

9 Features by Role **5**

10 Backend Connection **5**

11 License **5**

1 Project Status

- **Quality Gate:** [View Quality Gate Status]
- **Maintainability:** [Rating: A]
- **Security:** [Rating: A]
- **Reliability:** [Rating: A]
- **Technical Debt:** [Duplicated Lines: 1.5%]

2 Tech Stack

Technology	Version	Usage
Angular	21.2	Main framework (Standalone Components)
TypeScript	5.9	Development language
RxJS	7.8	Reactive programming
Bootstrap	5.3	Grid system and UI components
Bootstrap Icons	1.13	Iconography
Tailwind CSS	4.1	Additional CSS utilities
@ng-bootstrap	20.0	Bootstrap components for Angular
SweetAlert2	11	User alerts and modals
@stomp/rx-stomp	1.2	WebSocket client (STOMP protocol)
SockJS	1.6	WebSocket Fallback
ngx-cookie-service	21.1	Cookie management
Vitest	4.0+	Unit testing
Prettier	3.8	Code formatting
Node.js	20+	Runtime environment
npm	11+	Package manager

3 Prerequisites

- Node.js v20 or higher.
- npm v11 or higher.
- Spring Boot backend running (default at `http://localhost:8081`).

4 Installation

```
# 1. Clone the repository
git clone https://github.com/JaviRSDEV/FrontCarLog.git
cd FrontCarLog

# 2. Install dependencies
npm install
```

5 Environment Configuration

Environment files are ignored by Git. They must be created manually.

5.1 Local Development

Create `src/environments/environment.development.ts`:

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8081',
  wsUrl: 'ws://localhost:8081/ws-carlog',
};
```

5.2 Production

Create `src/environments/environment.ts`:

```
export const environment = {
  production: true,
  apiUrl: 'https://your-production-api.com',
  wsUrl: 'wss://your-production-api.com/ws-carlog',
};
```

6 Architecture

CarLog is built with **Angular 21** using **Standalone Components**. Each component manages its own dependencies via the `imports` array.

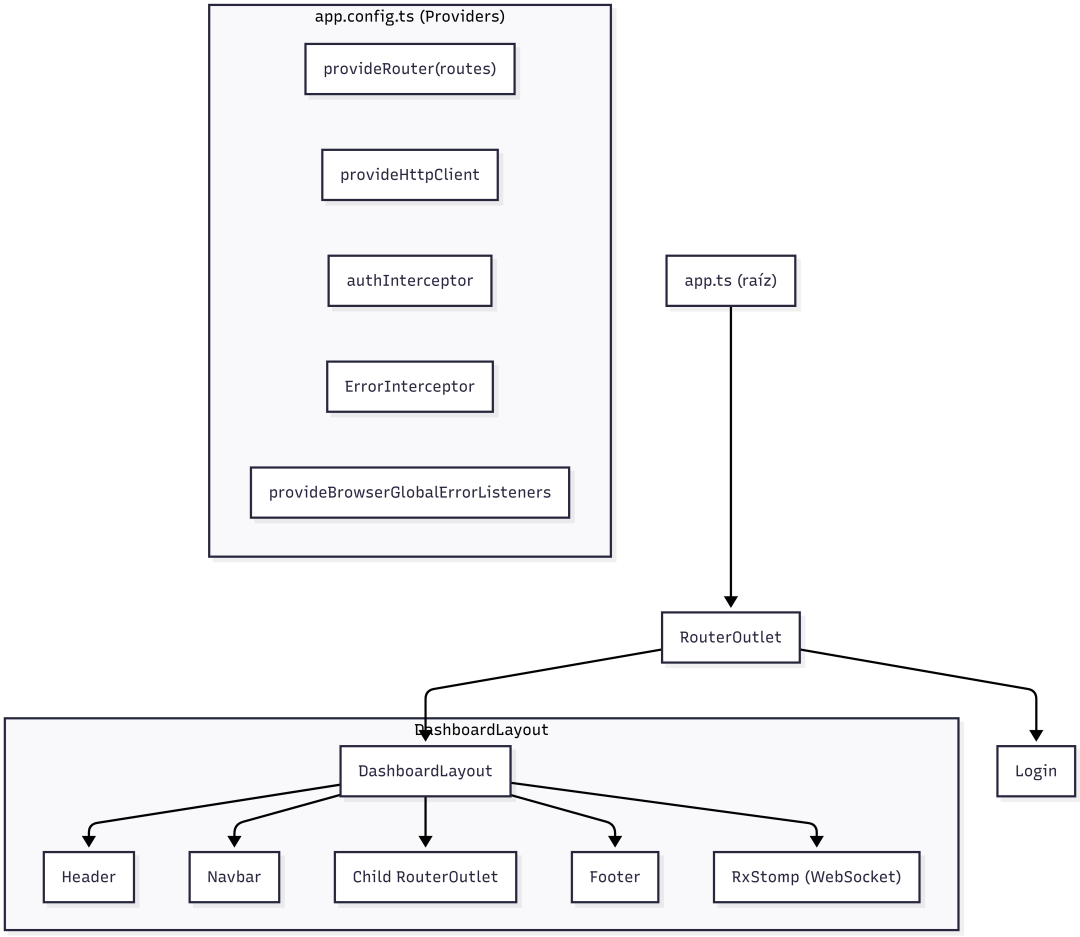

Figure 1: Component and provider architecture

7 Routes and Navigation

Route	Component	Guard(s)
`/`	`Login`	`loginGuard`
`/dashboard`	`DashboardLayout`	`authguardGuard`
`/dashboard/vehiculos`	`VehicleListComponent`	`tallerGuard`
`/dashboard/mantenimientos`	`WorkOrdersComponent`	`tallerGuard`
`/dashboard/taller`	`GestionTallerComponent`	`tallerGuard`
`/dashboard/alta-taller`	`AltaTaller`	`altaTallerGuard`
`**`	—	Redirects to `/dashboard`

8 Security and Authentication

Authentication is based on **JWT managed via secure HttpOnly cookies**.

8.1 Authentication Flow

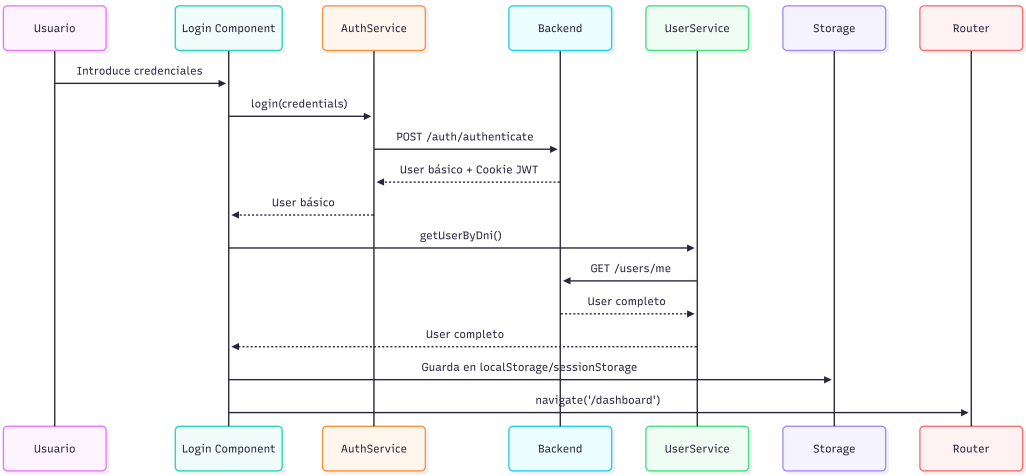

Figure 2: System authentication flow

9 Features by Role

Feature	MANAGER	CO_MANAGER	MECHANIC	CLIENT
Register workshop	✓	×	×	×
Edit workshop data	✓	✓	×	×
Invite / hire employees	✓	✓	×	×
View workshop fleet	✓	✓	×	×
View own vehicles	✓	✓	✓	✓
Create / edit vehicle	✓	✓	✓	✓
Update work order status	✓	✓	✓	×
Manage billing lines	✓	✓	✓	×

10 Backend Connection

CarLog Frontend consumes a Spring Boot REST API. All endpoints are prefixed with the URL defined in the environment configuration.

Domain	Base URL	Description
Authentication	`/auth`	Login, registration, logout
Users	`/users`	Profile and employee management
Vehicles	`/vehicles`	CRUD and history
Work Orders	`/workorders`	Order and billing management
WebSocket	`/ws-carlog`	Real-time notifications channel

11 License

This project is for academic and private use. Please consult the author for external permissions.